Cloud Computing (CS3081) Mini Project Report

Distributed Knowledge Distillation & High-Throughput Intent Inference on Kubernetes

Student: V Abhiram | Roll Number: 220103011 | Email: dimpuabhiram19@gmail.com | Repository: https://github.com/Abhi90111/cloud-intent-distiller | Release Tag: v1.0

1. Task 1: Object Storage and Parallel Teacher Labelling

Cluster Architecture & Persistent Object Storage: A local single-node Kubernetes cluster was instantiated using k3d from cluster/k3d.yaml. Decoupled object storage was realized via SeaweedFS deployed in the data namespace, attached to a 5 GiB PersistentVolumeClaim. The bootstrapping pipeline (scripts/bootstrap.sh) configured required namespaces (app, data, pipeline), generated secure access credentials in Kubernetes Secrets (s3-creds, seaweed-s3), initialized S3 buckets (datasets and models), and uploaded the primary command corpus.

Parallel Teacher Sharding: The 3,000 raw conversational prompts from commands_pool.jsonl were partitioned across parallel worker processes via deterministic modulo hashing: line_number % SHARD_COUNT == SHARD_INDEX. Two parallel batch Jobs (teacher-worker-shard0 and teacher-worker-shard1) executed in the pipeline namespace (manifested in k8s/worker-job.yaml, execution logs in results/pipeline_workers.txt). Worker 0 processed 1,500 even queries (outputs in results/teacher/shard-0.jsonl), while Worker 1 processed 1,500 odd queries (outputs in results/teacher/shard-1.jsonl), maintaining zero partition collisions or record loss.

Dataset Cleaning Audit: The dataset cleaning filter (worker/clean.py) screened all 3,000 generated responses against schema.json, removing syntax corruptions, unsupported intent categories, and slot hallucinations.

Dataset Audit Category	Evaluation Criteria	Records Filtered	Action Taken
(a) Malformed JSON	Syntax truncation or missing curly braces	4	Rejected
(b) Invalid Key Schema	Keys not strictly {intent, date, time, place}	7	Rejected
(c) Out-of-Domain Intent	Intent not in the 8 permitted categories	0	Rejected
(d) Hallucinated Slots	Extracted slot span not found verbatim in input query	81	Rejected
(e) Duplicate Commands	Repeated queries; lowest command ID retained	48	De-duplicated
Verified Cleaned Records	**Passed all validation filters into train.jsonl**	**2,860**	**Preserved**

Audit equation balances exactly: Input (3,000) = 4 + 7 + 0 + 81 + 48 + 2,860 = 3,000. Verified SHA-256 digest recorded in train.sha256.

2. Task 2: Training the Student (Sequence-Level Knowledge Distillation)

Model Training Setup: The Pythia-70M student model was fine-tuned on CPU on the initial 1,500 verified records of train.jsonl. Prompt token masking was applied: Command: {text}\nJSON: (conditioning prompt tokens masked with -100 to restrict loss calculation solely to predicted JSON target tokens). Hyperparameters in training/config.yaml: seed=3011, lr=0.0001, batch_size=8, epochs=3, max_len=128. Training executed via make train and validated using make train SUBSET=50.

Model Architecture	JSON Valid	Intent	Date	Time	Place	Full Frame
Untrained Pythia-70M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Teacher (Phi-3.5)	82.0%	59.0%	65.0%	76.0%	71.0%	39.0%
Student v1 (Deployed)	99.0%	74.0%	85.0%	74.0%	61.0%	29.0%
Student v2 (Greedy)	98.0%	68.0%	90.0%	98.0%	65.0%	35.0%

Conceptual Distillation Analysis:

(a) Sequence-Level Distillation Mechanics: Sequence-level distillation (Kim & Rush, 2016) trains the student on decoded discrete text generated by the teacher model. Decoded teacher predictions function as pseudo ground-truth target tokens, trained using standard autoregressive cross-entropy loss without necessitating soft teacher logit tensors.

(b) Logit-Based Distillation Principles: Logit-based distillation aligns continuous probability distributions by minimizing the Kullback-Leibler (KL) divergence between teacher and student temperature-softened softmax output vectors across the full vocabulary, imparting nuanced structural knowledge regarding unselected tokens.

(c) Vocabulary Incompatibility Constraints: Logit distillation demands an identical vocabulary space. Phi-3.5 operates on a 32,064-token vocabulary, whereas Pythia-70M uses the 50,304-token GPT-NeoX tokenizer. Because their subword tokenization boundaries and token IDs do not align, logit-level distribution comparison is mathematically ill-defined without an ad-hoc projection layer. Furthermore, Ollama only returns discrete decoded strings.

(d) Architectural Limitations: Sequence-level distillation propagates systematic teacher errors: when Phi-3.5 hallucinates non-existent slots (such as the 81 filtered cases), the student accepts them as valid supervision signals. Additionally, Pythia-70M's minimal parameter budget limits generalization across multi-clause voice inputs.

3. Task 3: The Inference Service in a Container

Microservice Implementation: The inference application is built using FastAPI (service/app.py) on Uvicorn listening on port 8000. It provides GET /healthz (200 OK while running), GET /readyz (503 Service Unavailable during weight pull/warmup, switching to 200 OK once loaded), and POST /parse. Headers X-Model-Version and X-Served-By accompany every HTTP response. Configuration is strictly environment-driven via MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS, and FAULT_INJECT, pulling weights securely from SeaweedFS S3 via credentials injected from Secret s3-creds.

Schema Validation & Fault Handling: Incoming client payloads are parsed via Pydantic CommandRequest, and generated predictions are checked against schema.json. Under FAULT_INJECT=1, synthetic malformed output ('not json') is injected, causing the service to return HTTP 500 with body

{"error": "invalid_model_output"}.

Resource Constraints & Peak Memory Profiling: Built on python:3.11-slim with CPU-only PyTorch (image size: 1.38 GB, well within the 2.0 GB budget). Fifty sequential parse requests sent to a container constrained to 1 CPU and 1 GiB RAM (results/docker_50req.txt) yielded 100% HTTP 200 responses, zero OOM events (OOMKilled=false), and peak memory usage of 468.2 MiB (45.72% of budget).

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

Deployment Topology & Ingress: Deployed in the app namespace with 2 initial replicas. Liveness probes inspect /healthz while readiness probes inspect /readyz. Ingress routing maps external /parse requests directly to Service parser. As captured in results/endpoints_during_startup.txt, the Service maintains <none> endpoints until model weights are fully initialized.

Autoscaling (HPA): HorizontalPodAutoscaler (k8s/hpa.yaml) targets 70% CPU utilization across 1-2 replicas. Under continuous Locust concurrency, aggregate CPU reached 195%, triggering scale-up from 1 to 2 replicas in under 45 seconds. Post-load cooldown (300 s window) smoothly scaled back to 1 replica (documented in results/hpa_watch.txt).

Failover, Recovery & Rollout: Deleting an active pod under live load triggered Kubernetes replica recreation in 17.5 seconds, with 450 requests seeing only 4 transient 500 errors (0.89% error rate, strictly beneath the 2.0% limit). Rolling update from v1 to v2 exhibited only 2 errors across 500 requests (0.40% error rate, well below the 0.5% ceiling), with kubectl rollout undo restoring v1 seamlessly.

Cold Start & Load Balancing: Container cold start to Ready state averaged a median of 7.1 s across 3 trials (runs: 8.2 s, 6.8 s, 7.1 s, evidence in results/cold_start_logs_run*.txt). Dual-replica traffic split balanced evenly: 172 requests to Pod A (50.6%) and 168 requests to Pod B (49.4%), comfortably surpassing the 30% per-pod minimum.

Local Cluster Environment Limitation: Single-node k3d runs within a containerized runtime on one host machine. While validating control plane scheduling, controller lifecycles, and horizontal scaling, it cannot replicate physical power loss or rack-level network partitioning.

5. Task 5: Subnetting and Network Access Control

VLSM Allocation Strategy (10.20.0.0/16): Subnets were allocated in descending order of usable host capacity: app (1,000 hosts needed -> /22 prefix, 1,022 usable hosts, 10.20.0.0 - 10.20.3.255); public (250 hosts needed -> /24 prefix, 254 usable hosts, 10.20.4.0 - 10.20.4.255); data (100 hosts needed -> /25 prefix, 126 usable hosts, 10.20.5.0 - 10.20.5.127); management (20 hosts needed -> /27 prefix, 30 usable hosts, 10.20.5.128 - 10.20.5.159). Full working in network/subnets.md.

Stateless NACL Configuration: Configured in network/nacl.json (inbound deny 50 for 198.51.100.0/24, allow 100 for HTTP 80, allow 150 for HTTPS 443, allow 200 for SSH from management, allow 250 for app reply traffic; outbound allow 50 to app port 8000, allow 100 for client replies on ephemeral ports 1024-65535). Verified 100% correct (6/6 packets) via python tools/nacl_sim.py network/nacl.json network/packets_sample.jsonl --check.

Stateless Mechanics & Rule Ordering: Stateless NACLs inspect each packet individually without state tables. Consequently, ephemeral reverse return traffic requires explicit bidirectional allowance. Placing block rule 50 ahead of general HTTP/HTTPS allow rules ensures malicious IP ranges are dropped prior to reaching broader port permissions.

6. Task 6: Reproducibility, Testing, and CI/CD

Single-Command Operations: The entire environment builds, provisions storage, imports images, and deploys services from a fresh clone via make up. Clean cluster teardown executes via make down.

Continuous Integration Pipeline (ci.yml): GitHub Actions pipeline executes 5 sequential stages: (1) code-linting with ruff; (2) unit-testing running 4 tests in tests/test_basic.py; (3) security-secret-scan with Gitleaks; (4) model-evaluation downloading model_v1.zip from GitHub Release v1.0 and running predict.py; and (5) quality-gate running score.py --min-validity 95.

Quality Gate Failure Verification: Triggering the pipeline with use_base: true runs inference with the untrained Pythia-70M baseline, producing 0% valid JSON and successfully triggering gate failure (exit status 1), as verified in CI workflow runs.

7. Task 7: Performance, Cost, and Review

Benchmark Results (Locust 4 users, 2 min, 2 replicas): 342 total requests, 2 failures (0.58% error rate, well under 1% limit). Median latency p50 = 1,250 ms, p95 = 2,700 ms (under 5,000 ms ceiling), throughput = 2.85 req/s. CPU load: 1928m and 1915m per pod. Memory: 742 MiB and 758 MiB. Load balanced: 172 / 168 requests (50.6% / 49.4%). Full metrics in results/loadtest_stats.csv and results/loadtest_headers.json.

Cost Estimation Arithmetic: Each replica is billed for 2 vCPU limit (2 x 3.50 = 7.00 INR/h) and 1 GiB memory limit (1 x 0.45 = 0.45 INR/h), totaling 7.45 INR/replica/h. Two replicas cost 14.90 INR/h. At 2.85 req/s (10,260 req/h), unit economic cost is: (14.90 INR / 10,260 req) * 1,000 = 1.45 INR per 1,000 requests.

SLO Formulation: Objective defined as >= 99% successful requests over one load-test run (342 requests). Permitted error budget = 342 x 0.01 = 3.42 requests. Actual failures observed = 2 requests (58.5% of budget used, SLO fully met at 99.42% success).

Comprehensive Architecture Review Table:

Component or Concern	Evidence (Measured Number)	Identified Limitation	Proposed Improvement
Data & Storage	7 files downloaded over cluster network at startup (median 7.1 s cold start)	Single-node SeaweedFS without high-availability or volume caching	Deploy replicated multi-node MinIO/Ceph; attach PVC node caches
Model Quality	2 of 342 outputs rejected as invalid JSON (0.58% error rate)	Compact 70M student exhibits occasional slot boundary errors	Incorporate larger distillation pool and grammar-constrained decoding
Latency & Throughput	p50 = 1,250 ms, p95 = 2,700 ms, 2.85 req/s on 2 replicas	CPU-bound inference; thread contention occurs if unconstrained	Apply INT8 ONNX Runtime quantization and continuous dynamic batching
Resource Utilization	CPU peaked at 1928m of 2000m limit; RAM at 758 MiB of 1 GiB	Pod capacity saturated near 100% under only 4 concurrent users	Right-size resource limits, scale horizontally across multi-node cluster
Reliability & Recovery	Pod failover restored Ready state in 17.5 s with only 4 errors (0.89%)	Single-node k3d cluster lacks true machine-level fault tolerance	Multi-node Kubernetes topology with PodDisruptionBudgets & anti-affinity
Security & Governance	S3 credentials in Secret s3-creds; unit cost is 1.45 INR / 1k reqs	Kubernetes Secret is only base64; Ingress lacks TLS encryption	Integrate HashiCorp Vault / External Secrets Operator and cert-manager TLS

8. Record of AI Tool Utilization

Antigravity (Gemini) and Anthropic Claude were utilized for architecture design, refactoring, Kubernetes manifests, and diagnosing PyTorch thread contention under CPU limits. Every suggested shell command, manifest, and Python refactor was executed locally and rigorously verified against automated testing suites. The stateless NACL was tested and verified with tools/nacl_sim.py (6/6 passing, 100%). Model scoring was audited with tools/score.py, and request timings were measured directly via Locust. AI inaccuracies (such as default Makefile target precedence and premature readiness probe pass) were identified, corrected, and documented. Distillation labels were produced exclusively by Phi-3.5 via Ollama without synthetic manual fabrication. Full verification logs are detailed in AI_USE.md.